GLPlot: GPU-accelerated interactive scientific visualization with a Matplotlib-like interface*

Juan Manuel Lombardi [1¶], Felix Riccius[1*], Julian Holland[1*], and Gianmarco Ducci[1*]

[1] *Fritz Haber Institute of the Max Planck Society, Berlin, Germany*

[¶] *Corresponding author: lombardi@fhi-berlin.mpg.de*

[*] *These authors contributed equally.*

16 August 2026

Summary

Interactive visualization is part of the scientific reasoning process: researchers use plots to inspect convergence, identify outliers, compare simulations, assess uncertainty, and decide which analyses or experiments to perform next. Furthermore, plots enable us to identify relationships that would otherwise be missed by visualizing abstract concepts alone; for these reasons, it is imperative that the scientific community have access to easy, fast tools to visualize their data. The complexity of these tasks rapidly escalates when a figure contains a large number of points, vertices, curves, or other geometric primitives. Conventional plotting workflows may incur substantial costs from CPU-side geometry construction, per-object draw calls, repeated rasterization, and transfer of expanded geometry to the graphics device. These speed bumps are not just a minor irritation, but an active hindrance to scientific discovery, as fewer potential lines of reasoning are pursued if one must wait hours to confirm one's logic.

Here, we propose `GLPlot`, an open-source Python plotting package that exposes a familiar `pyplot`-style interface while using an OpenGL rendering engine in the backend. It is intentionally developed to support common two- and three-dimensional scientific visualizations, including polylines, scatter plots, histograms, matrix images, contour and pseudocolor plots, vector fields, surfaces, and meshes. A more dedicated `plot_lines` interface represents large families of straight lines by their slope and intercept coefficients and expands their visible geometry analytically on the GPU.

[*] Software repository: https://github.com/AkarisDimitry/GLPlot

The rendering engine separates scene representation from GPU execution. It combines event-driven updates, shader-based geometry generation, stable viewport transformations, hardware clipping, density accumulation, adaptive level-of-detail control, and GPU-based picking. These operations are exposed to the user through a high-level Python interface, allowing large scientific datasets to be explored without direct interaction with low-level OpenGL components.

Statement of need

GPU-oriented visualization libraries can provide high rendering throughput, but they often require users to adopt a different programming model or work closer to graphics concepts than is desirable in routine scientific analysis (Campagnola et al., 2013; fastplotlib Developers, 2022). Conversely, pixel-based aggregation systems efficiently summarize very large datasets, but do not necessarily preserve the semantics or interactive selection of individual geometric objects (Datashader Developers, 2016; Stevens et al., 2015). `GLPlot` targets the space between these approaches. The intended users are computational scientists, engineers, and data analysts who require interactive rendering of large two- and three-dimensional datasets while retaining a concise interface close to the idiom of standard plotting libraries such as Matplotlib. The software is particularly suited to cases where the plotted entities remain meaningful geometric objects, such as line families in chemical-potential diagrams, feasibility diagrams, trajectory ensembles, vector fields, meshes, and selectable point clouds. The design is guided by five requirements:

1. avoid CPU-side geometry expansion when equivalent geometry can be generated analytically on the GPU;
2. maintain stable coordinate transforms during deep zoom operations at large coordinate offsets;
3. preserve density information when many primitives overlap;
4. avoid unnecessary CPU and GPU work when a scene is unchanged; and
5. support interactive inspection and selection without CPU searches that scale linearly with the number of rendered objects.

State of the field

Scientific visualization tools span a broad range of abstraction levels and rendering strategies. General-purpose plotting libraries prioritize flexibility and publication-quality output, GPU-oriented frameworks expose programmable rendering pipelines, and large-data visualization systems aggregate observations into display-resolution representations. These approaches address different requirements and therefore provide complementary rather than interchangeable solutions.

Matplotlib is the reference general-purpose visualization library for scientific

Python, supporting static, animated, and interactive figures across a broad range of plotting and publication workflows (Hunter, 2007). Its artist-based architecture and extensive layout, annotation, and backend systems make it appropriate for most conventional scientific figures. However, performance can deteriorate when a figure contains very large numbers of individual primitives, because geometry construction, artist management, coordinate transformation, and redraw operations are largely coordinated on the CPU. This limitation becomes particularly relevant for dense line families, large point clouds, trajectory ensembles, and complex three-dimensional scenes requiring repeated interactive updates.

VisPy provides high-performance interactive two- and three-dimensional visualization through an OpenGL-based rendering framework (Campagnola et al., 2013). Its interfaces range from relatively low-level graphics abstractions, including buffers, shaders, and rendering programs, to higher-level scene graphs and reusable visual components. This flexibility makes VisPy well suited to the development of custom visualization applications and specialized rendering techniques. At the same time, achieving highly specialized behavior may require users to work directly with graphics concepts or implement custom visual components, which can impose a substantial development burden for routine scientific plotting.

Datashader addresses large-data visualization through rasterization and aggregation into the finite pixel grid of the display (Datashader Developers, 2016). This strategy is highly effective when the primary objective is to preserve the statistical or spatial distribution of a dataset despite severe overplotting. Because the resulting representation is defined at the level of aggregate cells or image pixels, however, the original geometric objects are not necessarily retained as persistent elements of an interactive scene. This limits workflows in which individual lines, trajectories, points, or surfaces must remain separately identifiable, selectable, or subject to object-specific styling.

PyQtGraph is designed for scientific and engineering applications requiring responsive desktop graphics, data-acquisition displays, and rapidly updated signals (Campagnola & PyQtGraph Contributors, 2012). Its integration with Qt makes it particularly suitable for instrumentation, monitoring interfaces, and custom graphical applications. Its principal strengths lie in fast two-dimensional visualization and application development, whereas highly specialized three-dimensional rendering, analytical shader-side primitive generation, density accumulation, and large-scale GPU-based object selection are not its central design focus.

`GLPlot` targets the gap between these approaches. It retains a concise procedural interface close to Matplotlib's pyplot idiom while adopting a GPU-resident rendering architecture for large two- and three-dimensional scientific scenes. Its design combines analytical shader-side generation of specialized primitives, numerically stable coordinate transformations during deep zoom, density-aware rendering of overlapping geometry, suppression of unnecessary redraws, interaction caching, and GPU-based object picking. `GLPlot` is therefore intended for

workloads in which large numbers of geometric objects must remain individually represented, interactive, and semantically identifiable without requiring users to construct a custom graphics pipeline.

Software design

`GLPlot` converts plotting data into OpenGL-ready buffers on the CPU and delegates the expensive per-frame work to the GPU: 2D data is staged as float32 vertex buffers and rendered through shader-based wide-line and density-accumulation paths; 3D data is built into geometry buffers with model-view-projection transforms and rendered with perspective scaling, depth testing, ambient-occlusion/rim shading, and transparency composition. CPU readback is restricted to explicit export calls through `glReadPixels`; the interactive path never transfers pixels back to the host.

User-facing interface

The primary entry point is `glplot.pyplot`. Calls such as `plot`, `scatter`, `hist2d`, `pcolormesh`, `contour`, `quiver`, `plot_surface`, `mesh3d`, and `bar3d` create layers in a scene, accept NumPy arrays (Harris et al., 2020), and follow common Matplotlib calling conventions, including format strings for line and marker styling; select numerical routines use SciPy (Virtanen et al., 2020). A more specialized `plot_lines` interface instead takes a line family as coefficient pairs:

```
import numpy as np
import glplot.pyplot as plt

slopes = np.random.default_rng(0).normal(size=1_000_000)
intercepts = np.random.default_rng(1).normal(size=1_000_000)

plt.figure()
plt.plot_lines(slopes, intercepts, x_range=(-5.0, 5.0))
plt.set_cmap("magma")
plt.density_gain(0.8)
plt.xlabel("x")
plt.ylabel("a x + b")
plt.show(density=True)
```

This data model reduces host-side geometry construction: rather than the CPU generating per-line endpoints, the active viewport determines each line's visible segment analytically in the vertex shader.

Rendering architecture and precision

Data arrays, styles, coordinate transforms, and renderer-specific metadata are stored as scene layers; a rendering manager maps each layer to a primitive

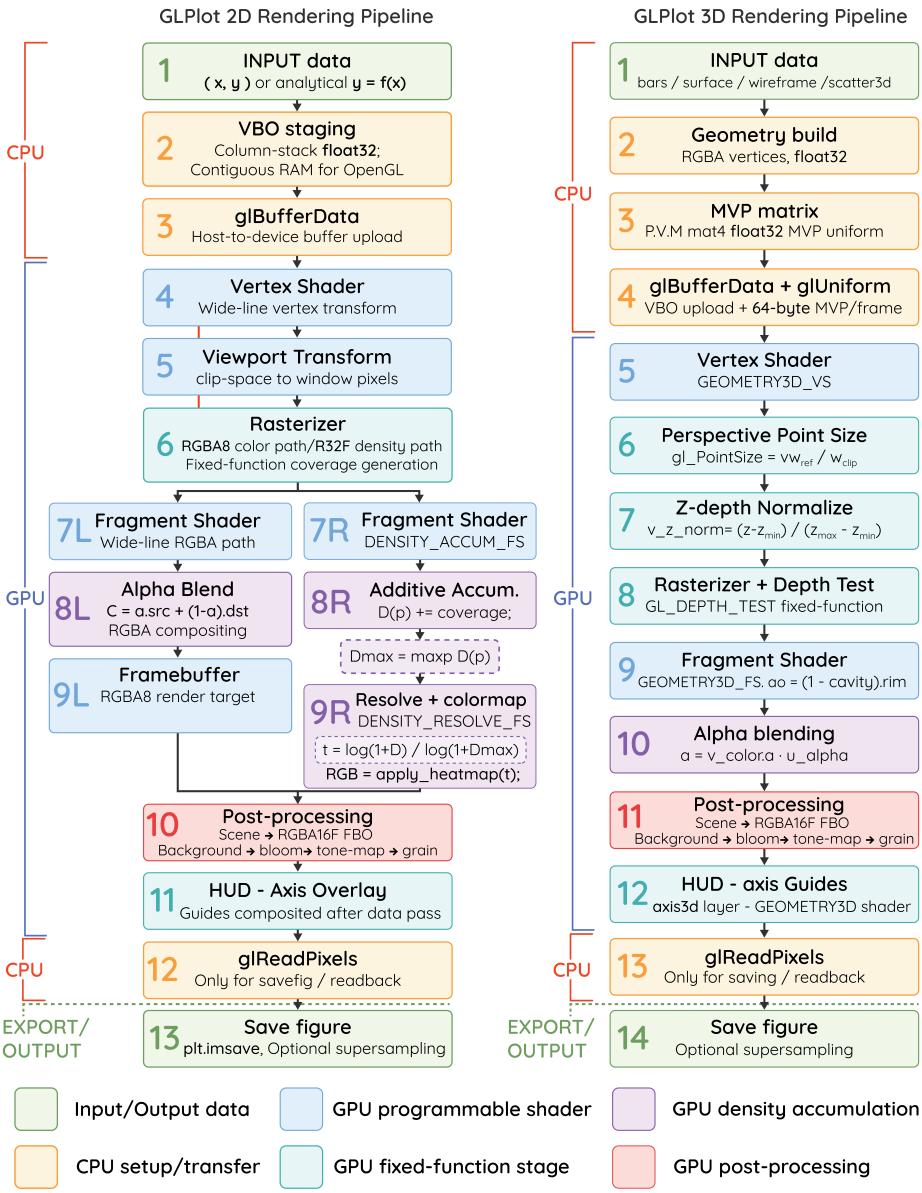

Figure 1: GLPlot's 2D and 3D rendering pipelines, from CPU-side data staging through GPU shading, blending, and post-processing to optional export.

renderer and the framebuffer passes it needs. The render loop is reactive, redrawing only when a layer, camera transform, or effect is marked dirty, so an unchanged figure costs nothing between events.

For a line family $y_i(x) = a_i x + b_i$, `GLPlot` uploads (a_i, b_i) as instanced attributes; the vertex shader analytically recomputes each line's visible endpoints against the current viewport bounds and discards out-of-domain instances before rasterization, rather than the CPU generating explicit per-line geometry. To stay numerically stable during deep zoom, the viewport center is computed in double precision on the CPU, and world coordinates are expressed relative to it before conversion to normalized device coordinates — keeping the values the GPU sees close to the display range regardless of the absolute coordinate offset.

Density, interaction, and level of detail

Dense, overlapping geometry is accumulated additively into a floating-point framebuffer, log-normalized to the display range, and optionally tone-mapped in the style of Reinhard et al. (2002) to preserve contrast. During rapid pan or zoom, a cached offscreen image is reprojected over the expanded viewport and an exact redraw is scheduled once interaction stops, so pointer motion never forces full geometry work. Level of detail is decided from estimated fill-rate cost rather than primitive count alone, using viewport size, line width, and projected polyline length; because the decision is derived from the instance identifier, a static view does not flicker between frames. Object selection uses a deferred integer picking pass: selectable primitives write an encoded identifier to an integer framebuffer, and a click resolves to a layer and element index from a single-pixel readback, independent of scene size.

Research impact

`GLPlot` is designed for research workflows in which visualization is a bottleneck between numerical computation and interpretation. It has been used in this capacity in `EZGA`, a published evolutionary structure-exploration framework for computational materials science, to visualize energy landscapes, convex-hull reconstructions, and surface phase diagrams produced across its benchmark systems (Lombardi et al., 2026). More broadly, representative applications include the inspection of large trajectory projections, chemical-potential line families, phase-stability or feasibility constructions, energy distributions, structural descriptors, spatial fields, large vector fields, and uncertainty ensembles. Related visualization patterns occur in atomistic simulation, computational chemistry and physics, bioinformatics, engineering simulation, and exploratory data analysis.

The principal scholarly contribution is the integration of multiple GPU rendering techniques into a coherent scientific plotting interface: analytical primitive expansion, viewport-relative transforms, hardware clipping, floating-point density

accumulation, interaction caching, fill-rate-aware level of detail, lightweight depth cues for three-dimensional primitives, and integer-ID picking. Although these techniques are established individually in computer graphics, their combination is intended to make them usable by researchers who should not need to manage shaders and framebuffer objects directly.

Limitations

`GLPlot` focuses on high-performance interactive visualization rather than exhaustive Matplotlib compatibility. Complex publication layouts, specialized axis artists, advanced tick formatting, and some annotation or backend features may remain better served by Matplotlib. GPU rendering also introduces hardware, driver, and context-creation dependencies. Reproducible performance reports should therefore record the GPU model, driver and OpenGL versions, operating system, Python version, package version, viewport size, and rendering configuration.

The package currently requires a functional OpenGL context and a driver supporting OpenGL 3.3 core profile, the minimum version its shaders and context request. The continuous integration suite runs fully headless on this same core-profile context, without opening a visible window, so it also serves as a compatibility check for reviewer testing and automated report generation.

AI usage disclosure

Portions of `GLPlot`'s implementation, its test suite, its packaging configuration, and this manuscript (including this disclosure) were produced with assistance from Claude (Anthropic), an AI coding assistant, across multiple development sessions. The author directed the work, formulated the problems to be solved, reviewed and tested the AI-assisted output, and is responsible for all final decisions, correctness, and content in the software and this paper.

References

Campagnola, L., Klein, A., Larson, E., Rossant, C., & Rougier, N. P. (2013). *VisPy: Interactive scientific visualization in Python*. https://vispy.org.

Campagnola, L., & PyQtGraph Contributors. (2012). *PyQtGraph: Scientific graphics and GUI library for Python*. https://www.pyqtgraph.org.

Datashader Developers. (2016). *Datashader: Accurate, fast visualization of large datasets*. https://datashader.org.

fastplotlib Developers. (2022). *fastplotlib: Next-generation GPU-accelerated scientific visualization*. https://github.com/fastplotlib/fastplotlib.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R.,

Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Lombardi, J. M., Riccius, F., Pare, C. W. P., Heenen, H. H., Reuter, K., & Scheurer, C. (2026). EZGA: An evolutionary structure exploration framework. *Computer Physics Communications*, 110354. https://doi.org/10.1016/j.cpc.2026.110354

Reinhard, E., Stark, M., Shirley, P., & Ferwerda, J. (2002). Photographic tone reproduction for digital images. *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '02)*, 267–276. https://doi.org/10.1145/566570.566575

Stevens, J.-L. R., Rudiger, P., & Bednar, J. A. (2015). HoloViews: Building complex visualizations easily for reproducible science. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 59–66. https://doi.org/10.25080/Majora-7b98e3ed-00a

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2